Java aktuell

iJUG member
java-aktuell.eu

Java & Architektur

OpenJDK 26/27, Spring Boot Best Practices, HTMX und moderne Frontend-Strategien

KI & Verantwortung

LLMs und APIs, Bias in neuronalen Netzen, Architektur im KI-Zeitalter

Menschen & Software

Psychologie in Projekten, Wirksamkeit von Entwickler:innen, Informatik-Nachwuchs fördern



Ausgabe 04/2026

Java aktuell

Praxis. Wissen. Networking. Das Magazin für Entwicklerinnen und Entwickler

D: 7,95 EUR | CH: 11,99 CHF | A: 9,00 EUR | Benelux: 8,50 EUR

4 191978 307959 04

Liebe Leserinnen und Leser,

in dieser Ausgabe erwartet euch erneut eine abwechslungsreiche Mischung aus Technologie, Methodik, Architektur und den Auswirkungen von KI auf unseren Berufsalltag. Den Auftakt gibt wieder einmal Andreas Badelt mit seinem Java-Tagebuch, das euch auf den neuesten Stand rund um alle Geschehnisse innerhalb der Java-Community bringt. Im Anschluss folgt Andreas Monschau, der seine satirische Reihe über das zuverlässige Scheitern von Softwareprojekten fortsetzt und zeigt, warum ungeprüfte Biases und Denkfehler in Projekten oft ein unterschätztes Risiko darstellen.

Anschließend wirft Falk Sippach ab Seite 14 einen Blick auf die Neuerungen von OpenJDK 26 und 27. Ihr erfahrt, welche Entwicklungen die Java-Plattform in den Bereichen Performance, Nebenläufigkeit, Sicherheit und moderne Schnittstellen voranbringen. Mit dem Zusammenspiel von KI und Schnittstellen beschäftigen sich Benny Burkert und Felix Medam, die erläutern, warum LLMs beim Konsumieren von HTTP-APIs mehr benötigen als technische Erreichbarkeit und eine OpenAPI-Beschreibung allein.

Für die tägliche Entwicklungspraxis liefert Julius Mischok ab Seite 32 zehn pragmatische Empfehlungen für langfristig stabile und wartbare Spring-Boot-Anwendungen. Danach zeigen Sebastian Sprenger und Felix Seidel, wie sich mit HTMX und einem backendzentrierten Ansatz moderne Anwendungen auch mit deutlich weniger JavaScript umsetzen lassen. Passend dazu untersucht Emil Pawlowski, wie viel Verantwortung ein Client überhaupt tragen sollte, und vergleicht verschiedene Ansätze vom klassischen HTML bis zur React-basierten Single-Page-Application.

Einen wichtigen Schwerpunkt bildet zudem das Thema Künstliche Intelligenz. Eldar Sultanow und Lars Santesson beleuchten in ihrem Artikel ab Seite 52, wie sich die Rolle von IT-Architektinnen und -Architekten im KI-Zeitalter verändert. Dr. Sandra C. Signore zeigt anschließend, wie neuronale Netze gesellschaftliche Verzerrungen übernehmen können und welche Verantwortung Entwicklerinnen und Entwickler entlang des gesamten KI-Lebenszyklus tragen.

Abgerundet wird diese Ausgabe durch zwei Beiträge, die den Menschen in den Mittelpunkt stellen: Christian Seifert geht der Frage nach, warum sich viele Entwicklerinnen und Entwickler im Projekttalltag trotz hoher Kompetenz oft ohnmächtig fühlen und wie sie wieder mehr Wirksamkeit entfalten können. Zum Abschluss berichtet Fin Labusch von zehn Jahren IT-Workshops für Kinder und gibt Einblicke, wie ihr junge Menschen spielerisch für Informatik und Programmierung begeistern könnt.

Wir wünschen euch viel Freude beim Lesen und viele Impulse, die euch inspirieren, motivieren, weiterbringen und zum Nachdenken anregen.





Lisa Damerow
Redaktionsleitung Java aktuell

INHALT



Java 26 und 27:
Die Releases im Detail



LLMs und APIs:
Warum OpenAPI allein nicht reicht

3 Editorial

6 Java-Tagebuch

10 Wie du deine Softwareprojekte zuverlässig ruinierst – Teil 3: Nehmt jedes Bias mit (ohne es zu hinterfragen)
Andreas Monschau

14 OpenJDK 26 und 27:
Die Neuerungen bei Java in 2026
Falk Sippach

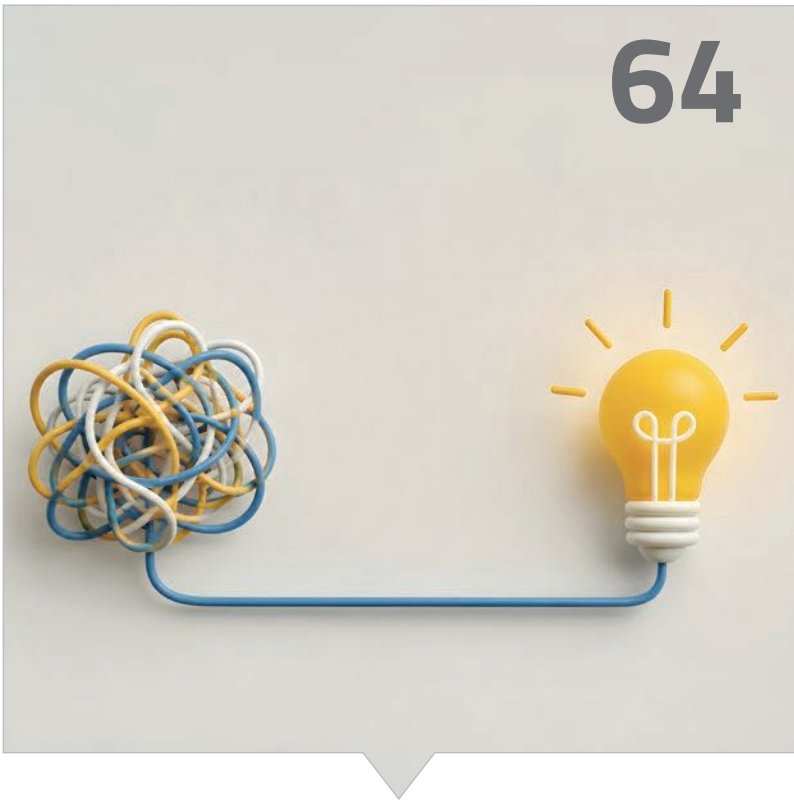
22 Wenn LLMs HTTP-APIs konsumieren: Qualität, Kontext und ausführbare Capabilities
Benny Burkert & Felix Medam

32 10 Tipps für Spring Boot:
Weniger Magie, mehr Pragmatismus
Julius Mischok

38 Dank HTMX zu einem Backend-lastigen Frontend
Sebastian Sprenger & Felix Seidel



Bias in KI-Systemen:
Wenn Algorithmen Vorurteile lernen



Vom Frust zur Wirksamkeit: Wie gute Entwickler:innen wieder mehr Einfluss gewinnen

44 Mehr als nur Single-Page-Applications: Wie viel Verantwortung braucht der Client wirklich?
Emil Pawlowski

52 Die neue Rolle des Architekten im KI-Zeitalter
Eldar Sultanow & Lars Santesson

58 Fairness not found: Wie neuronale Netze gesellschaftliche Verzerrungen lernen
Dr. Sandra C. Signore

64 Zwischen Frust und Verantwortung: Warum gute Entwickler sich ohnmächtig fühlen – und wie sie Wirksamkeit zurückgewinnen
Christian Seifert

70 Was macht ein Java-Entwickler in seiner Freizeit? Oder: 10 Jahre IT-Workshops für Kinder
Fin Labusch

78 Impressum/Inserenten

OpenJDK 26 und 27: Die Neuerungen bei Java in 2026

Falk Sippach, embarc Software Consulting GmbH




26
27

Mit Java 26 und Java 27 sind in diesem Jahr wieder zwei neue Versionen der Java-Plattform erschienen. Das OpenJDK 26 wurde im März veröffentlicht, das OpenJDK 27 folgte im September. Viele Änderungen zahlen auf langfristige Themen ein: bessere Start- und Laufzeitperformance, reifere Nebenläufigkeit, mehr Plattformintegrität und modernere Schnittstellen für Web und Security. In diesem Artikel werfen wir wieder einen Blick auf die Neuerungen.

Beide Releases enthalten zusammen 19 JEPs – wobei ein erheblicher Teil davon bereits bekannte Preview- und Inkubator-Features weiterentwickelt. Für einen ersten Überblick ist wie immer die Projektseite des OpenJDK *[1]* ein guter Startpunkt. Zehn JEPs (JDK Enhancement Proposals) sind dort bei Java 26 *[2]* gelistet:

- 500: Prepare to Make Final Mean Final
- 504: Remove the Applet API
- 516: Ahead-of-Time Object Caching with Any GC
- 517: HTTP/3 for the HTTP Client API
- 522: G1 GC: Improve Throughput by Reducing Synchronization
- 524: PEM Encodings of Cryptographic Objects (Second Preview)
- 525: Structured Concurrency (Sixth Preview)
- 526: Lazy Constants (Second Preview)
- 529: Vector API (Eleventh Incubator)
- 530: Primitive Types in Patterns, instanceof, and switch (Fourth Preview)

Java 27 *[3]* bringt neun weitere beziehungsweise erneut aufgenommene JEPs:

- 523: Make G1 the Default Garbage Collector in All Environments
- 527: Post-Quantum Hybrid Key Exchange for TLS 1.3
- 531: Lazy Constants (Third Preview)
- 532: Primitive Types in Patterns, instanceof, and switch (Fifth Preview)
- 533: Structured Concurrency (Seventh Preview)
- 534: Compact Object Headers by Default
- 536: JFR In-Process Data Redaction
- 537: Vector API (Twelfth Incubator)
- 538: PEM Encodings of Cryptographic Objects (Third Preview)

Das sind diesmal deutlich weniger JEPs als in den letzten Releases. Es setzt sich jedoch der Trend fort, Java durch gezielte Verbesserungen moderner, effizienter und leistungsfähiger zu machen. Neben Änderungen an der Sprache gibt es auch Optimierungen in der JVM, neue APIs sowie die Unterstützung neuer Protokollversionen. Besonders auffällig ist, dass mehrere Preview-Features weiter geschärft, statt vorschnell finalisiert zu werden. Die Java Releases im Jahr 2026 erscheinen damit zwar weniger spektakulär, legen aber wichtige Grundlagen für die nächsten Jahre.

Pattern Matching nähert sich auch den primitiven Typen

Pattern Matching gehört seit mehreren Jahren zu den sichtbarsten Sprachentwicklungen in Java. Nach Type, Record sowie Unnamed Patterns und Pattern Matching für switch soll das Konzept auch auf primitive Typen ausgedehnt werden. Primitive Werte sollen künftig nach denselben Regeln behandelt werden wie Referenztypen. So lässt sich natürlich schon immer ein long-Wert direkt in einem switch verwenden *(siehe Listing 1)*.

```
static String classify(long value) {
     return switch (value) {
           case 0L -> "zero";
           case 1L -> "one";
           default -> "many";
     };
}
```

Listing 1: Constant Patterns mit primitiven Werten im switch

Interessanter ist jedoch die Verbindung von Pattern Matching und primitiven Konvertierungen. So können jetzt neben Referenztypen auch primitive Typen als Patterns eingesetzt werden oder bei Record Patterns auch kombiniert werden *(siehe Listing 2)*. Dadurch werden Konvertierungen, instanceof und switch in ein konsistenteres Typsystem integriert. Java 26 hat mit JEP 530 die vierte Preview gebracht. Unter anderem wurden die Regeln für unbedingt exakte Konvertierungen und die Dominanz von case-Labeln weiter präzisiert. Java 27 hat das Feature mit JEP 532 in der fünften Preview unverändert erneut vorgelegt. Dass nur noch wenige Änderungen notwendig sind, deutet darauf hin, dass sich das Feature seiner endgültigen Form nähert. Wie im Beispiel in *Listing 3* zu sehen ist, können Entwickler künftig auch einfach prüfen, ob ein ganzzahliger Wert in den Wertebereich eines Byte-Werts passt.

```
record Measurement(String sensor, long value) {}
var m = new Measurement("temperature", 25);
if (m instanceof Measurement(String sensor, int value)) {
     IO.println(sensor + ": " + value);
}
```

Listing 2: Record Patterns mit primitiven Typen

```
String checkByte(int value) {
  if (value instanceof byte b) {
     return "byte b = " + b;
  }
  return "kein byte: " + value;
}
IO.println(checkByte(127)); // b = 127
IO.println(checkByte(128)); // kein byte: 128
```

Listing 3: Prüfung auf Wertebereiche von primitiven Datentypen

Structured Concurrency: Die API wird weiter geschärft

Mit Virtual Threads hat Java ein leichtgewichtigeres Thread-Modell bekommen. Dabei wird aber nur die Frage beantwortet, wie güns-

tig viele nebenläufige Tasks ausgeführt werden können. Es beantwortet jedoch nicht automatisch die Frage, wie deren Lebenszyklus, Fehler und Ergebnisse strukturiert werden. Hier setzt Structured Concurrency an. Mehrere fachlich zusammengehörige Tasks werden in einem `StructuredTaskScope` zusammengefasst *(siehe Listing 4)*.

```
try (var scope = StructuredTaskScope.open()) {
     var user = scope.fork(this::loadUser);
     var orders = scope.fork(this::loadOrders);
     scope.join();
     return new Result(user.get(), orders.get());
}
```

Listing 4: StructuredTaskScope startet zwei Aufgaben

Die gestarteten Tasks können den Scope nicht einfach unkontrolliert überleben. Abbruch, Fehlerbehandlung und Lebensdauer werden Teil der Kontrollstruktur. Im Jahr 2026 ging Structured Concurrency in zwei weitere Preview-Runden. Java 26 bringt die sechste, Java 27 die siebte Preview. Dabei hat sich vor allem die Joiner-API verändert. Ein Joiner beschreibt, wann eine Gruppe von Tasks als beendet gilt und welches der Ergebnisse relevant ist. Beispielsweise liefert `Joiner.allSuccessfulOrThrow()` die Ergebnisse nur dann, wenn alle Tasks erfolgreich waren. Für ein Rennen mehrerer Tasks liefert `Joiner.anySuccessfulOrThrow()` das erste erfolgreiche Ergebnis zurück. In Java 27 wurde die Fehlerbehandlung weiter ausgebaut. Joiner können nun expliziter bestimmen, welche Art von Exception geworfen wird. Neue überladene Methoden wie `Joiner.allSuccessfulOrThrow(exceptionMapper)` erlauben zudem eine anwendungsbezogene Übersetzung von Fehlern. Auch die Timeout-Behandlung wurde überarbeitet. Java 27 hat unter anderem `CancelledByTimeoutException` eingeführt und `onTimeout()` zu `timeout()` umbenannt.

Die ungewöhnlich hohe Zahl der Preview-Runden sollte nicht darüber hinwegtäuschen, dass die Grundidee inzwischen stabil ist. Die Arbeit konzentriert sich inzwischen vor allem darauf, eine API zu finden, die typische Fälle einfach ausdrückt und trotzdem genügend Flexibilität für Fehlerbehandlung, Abbruch und Timeouts bietet.

Lazy Constants: verzögert initialisieren, anschließend unveränderlich

Eine weitere Lücke im Java-Programmiermodell betrifft die verzögerte Initialisierung von final-Attributen. Da ein final-Feld unveränderlich ist, muss der Wert unmittelbar initialisiert werden. Ist diese Initialisierung teuer und soll erst beim ersten tatsächlichen Zugriff geladen werden, benötigte man bisher eigene, nicht trivial zu implementierende Lazy-Initialization-Mechanismen. Hier setzen Lazy Constants an. Sie sind eine Art Wrapper, deren Inhalt beim ersten Zugriff einmalig und threadsicher berechnet und anschließend lesend zur Verfügung steht *(siehe Listing 5)*.

```
static final LazyConstant<Configuration> CONFIG =
     LazyConstant.of(Configuration::load);
[..]
// Initialisierung erfolgt verzögert beim ersten Zugriff
CONFIG.get();
```

Listing 5: Erzeugung einer LayzConstant und spätere Initialisierung

Das Feature begann bereits in Java 25 unter dem Namen Stable Values. Mit Java 26 wurde die API deutlich umgebaut und das Konzept in Lazy Constants umbenannt. Die ursprünglichen Möglichkeiten, einen Wert explizit über Methoden wie `trySet()` oder `setOrThrow()` zu setzen, sind verschwunden. Stattdessen wird die Initialisierungsfunktion bereits beim Anlegen des Lazy Constants angegeben. Das macht die Abstraktion wesentlich klarer: Es handelt sich nicht um eine einmal beschreibbare Variable, sondern um eine Konstante, deren Erzeugung aufgeschoben wird.

Es lassen sich auch Lazy Collections (Listen und Maps) auf diese Art definieren. Die einzelnen Felder bzw. Elemente werden erst erzeugt,

```
enum Option { VERBOSE, DRY_RUN, STRICT }

// Return true when the given Option is enabled
boolean isEnabled(Option option) {
     IO.println(String.format("isEnabled(%s) ...", option.name()));
     // could be read from a configuration file (expensive)
     if (option == Option.DRY_RUN) {
         return false; // DRY_RUN is disabled
     }
     return true;
}

// Lazily initialized Set of Options
final Set<Option> OPTIONS =
     Set.ofLazy(EnumSet.allOf(Option.class), this::isEnabled);

void main() {
     IO.println("started ...");
     if (OPTIONS.contains(Option.DRY_RUN)) {
           IO.println("dry run mode enabled");
     }
     if (OPTIONS.contains(Option.STRICT)) {
           IO.println("strict mode enabled");
     }
     // Actual processing logic
}
```

Listing 6: Lazy Set mit verzögerter Initialisierung der Werte

wenn tatsächlich darauf zugegriffen wird. Java 27 ergänzt schließlich `Set.ofLazy(...)` *(siehe Listing 6)*. Gleichzeitig sind `isInitialized()` und `orElse()` verschwunden. Denn der Initialisierungszustand soll bewusst nicht Teil der sichtbaren Abstraktion sein. Der Benutzer soll eine Lazy Constant wie einen normalen unveränderlichen Wert behandeln. Damit geht es bei Lazy Constants um mehr als nur syntaktischen Zucker. Die JVM weiß nach erfolgreicher Initialisierung, dass sich der Wert nicht mehr ändern wird und kann diese Information für Optimierungen nutzen.

HTTP/3 erreicht den Java HTTP Client

Der mit Java 11 eingeführte HTTP-Client wurde in Java 26 um die Enum-Konstante `HttpClient.Version.HTTP_3` ergänzt. Ein Client kann damit HTTP/3 bevorzugen *(siehe Listing 7)*. Am grundsätzlichen Programmiermodell ändert sich nichts. Requests werden weiterhin wie gewohnt erzeugt und versendet.

```
var client = HttpClient.newBuilder()
            .version(HttpClient.Version.HTTP_3)
            .build();
```

Listing 7: Aktivierung von HTTP/3

HTTP/3 verwendet im Gegensatz zu HTTP/1.1 und HTTP/2 nicht TCP, sondern QUIC über UDP. Einer der Vorteile besteht darin, dass Paketverluste nicht mehr sämtliche parallel laufende HTTP-Streams einer Verbindung blockieren. HTTP/3 wird allerdings nicht zum neuen Default. Der HTTP-Client bevorzugt weiterhin HTTP/2. Wer HTTP/3 verwendet, muss es bewusst auswählen. Unterstützt der Server das Protokoll nicht, kann der Client auf HTTP/2 oder HTTP/1.1 zurückfallen. Das OpenJDK beschränkt sich dabei bewusst auf HTTP/3. Eine allgemeine öffentliche QUIC-API gehört nicht zum JEP.

G1 wird schneller – und schließlich überall Standard

Auch bei der Garbage Collection gibt es wieder kleine Änderungen. Der G1 ist bereits seit Java 9 der Standard-Garbage-Collector, wird aber weiterhin intensiv optimiert. JEP 522 reduziert in Java 26 die Synchronisation zwischen Anwendungs- und GC-Threads. Bestimmte Barrier-Operationen mussten bislang globale Datenstrukturen aktualisieren, was bei vielen Threads zu Konflikten führen konnte. Die neue Implementierung verwendet stärker thread-lokale Strukturen und soll dadurch insbesondere den Durchsatz von Anwendungen mit vielen aktiven Threads verbessern.

Und Java 27 geht noch einen Schritt weiter. Denn bislang war die Aussage „G1 ist der Default-GC" nicht ganz richtig. Auf sehr kleinen Systemen, insbesondere bei nur einer verfügbaren CPU oder sehr wenig Arbeitsspeicher, konnte die HotSpot-JVM stattdessen automatisch den Serial GC auswählen. JEP 523 entfernt diese Sonderregel und ab Java 27 ist der G1 unabhängig von der erkannten Umgebung der Standard. Der Serial GC verschwindet nicht und kann weiterhin explizit gewählt werden *(siehe Listing 8)*. Relevant ist die

```
java ... -XX:+UseSerialGC
```

Listing 8: Serial GC beim Start aktivieren

Änderung vor allem bei kleinen (Docker-) Containern. Bisher konnte schon eine Änderung der CPU- oder Memory-Limits indirekt dazu führen, dass eine Anwendung mit einem anderen Garbage Collector startete. Dieses Verhalten wird nun vorhersehbarer.

Kleinere Objekt Header werden Standard

Auch Compact Object Headers sind keine völlig neue Technik. Sie stammen aus Project Lilliput und wurden bereits in vorherigen Java-Versionen eingeführt und erprobt. Ein Java-Objekt benötigt auf einer typischen 64-Bit-JVM zusätzlich zu seinen Nutzdaten einen Header. Mit komprimierten Class Pointern waren dies typischerweise zwölf Byte. Compact Object Headers reduzieren diesen Header auf acht Byte. Die Ersparnis von vier Byte erscheint zunächst wenig. Bei Anwendungen mit Millionen kleiner Objekte summiert sich der Unterschied jedoch erheblich. Zusätzlich passen mehr Objekte in die CPU-Caches und der Garbage Collector muss weniger Speicher verarbeiten. Erste Erfahrungen (zum Beispiel bei Anwendungen auf AWS) zeigen, dass sich der Heap-Verbrauch um zirka 10 bis 20 % reduziert und der Durchsatz um zirka 5 bis 10 % erhöht.

Java 25 machte Compact Object Headers zu einem regulären, aber noch optionalen Feature. Mit dem JEP 534 werden sie ab Java 27 standardmäßig aktiviert. Wer das alte Layout benötigt, kann vorerst zurückschalten *(siehe Listing 9)*. Für normalen Java-Code ist die Änderung transparent. Interessant wird sie vor allem für JVM-nahe Tools und Bibliotheken, die Annahmen über das konkrete Objektlayout treffen.

```
java ... -XX:-UseCompactObjectHeaders
```

Listing 9: Compact Object Headers deaktivieren

Project Leyden arbeitet weiter an Startzeit und Warmup

Ein weiteres Performance-Thema ist das Ahead-of-Time (AOT) Caching aus Project Leyden. Die Idee besteht darin, die Zeit, welche die JVM zum Warmwerden nach dem Start einer Anwendung braucht, in einen vorherigen Trainingslauf zu verlagern. Informationen über geladene Klassen, Objekte und Profilinformationen können anschließend in einem AOT-Cache gespeichert und bei späteren Starts wiederverwendet werden. Bislang gab es dabei ein Problem: Im Cache abgelegte Java-Objekte waren teilweise an das konkrete Speicherlayout des Garbage Collectors gebunden. Insbesondere ZGC verwendet intern andere Repräsentationen als G1, Serial oder Parallel GC. Der JEP 516 (Ahead-of-Time Object Caching with Any GC) beseitigt diese Einschränkung. Objekte können nun in einem GC-neutralen Format gespeichert und beim Start entsprechend dem tatsächlich verwendeten Garbage Collector materialisiert werden. Damit müssen sich Anwendungen nicht mehr zwischen den Startup-Vorteilen des AOT-Cache und einem Low-Latency-Collector wie ZGC entscheiden. Diese Neuerung ist vor allem ein weiterer Baustein von Project Leyden. Es wird darauf hingearbeitet, immer mehr Arbeit aus Startup und Warmup vorzuverlegen, ohne Java in ein statisches Native-Image-Modell wie bei GraalVM zu verwandeln.

Security: von PEM bis Post Quantum

Java hat seit jeher einen starken Fokus auf Security-Funktionen. Im Moment wird die Sprache insbesondere auf die Zukunft der

Quantencomputer vorbereitet, denn dann sind heutige Verschlüsselungsstandards nicht mehr ausreichend. Auch für im Sicherheitsumfeld relevante Formate werden APIs entwickelt. Gerade mit PEM-Dateien kommen Entwickler häufig in Berührung *(Listing 10)*. Trotzdem hatte Java lange Zeit keine einheitliche API dafür. Daher mussten Entwickler selbst Header entfernen, Base64 dekodieren und anschließend mit verschiedenen Klassen wie CertificateFactory, KeyFactory und KeySpec kombinieren.

```
-----BEGIN CERTIFICATE-----
...
-----END CERTIFICATE-----
```

Listing 10: PEM File

Im OpenJDK 25 wurde die neue PEM API mit den Klassen PEMEncoder und PEMDecoder als Preview vorgestellt. In Java 26 und 27 erschienen die zweite und dritte Preview. Dabei wurde die API nochmals strukturell überarbeitet, unter anderem wurde die bisherige Abstraktion DEREncodable durch die allgemeinere BinaryEncodable ersetzt. Zertifikate, Key Pairs und verschiedene Key Specs implementieren anschließend diese Schnittstelle. Auch PEMEncoder wurde entsprechend angepasst. Der Java Almanac *[4]* zeigt die Veränderungen der dritten Preview detailliert. Die Grundidee bleibt aber stabil: Das JDK soll PEM als eigenes kryptografisches Format verstehen, statt Anwendungen dazu zu zwingen, PEM lediglich als Base64-Text zu behandeln.

Noch grundlegender ist JEP 527 (Post-Quantum Hybrid Key Exchange for TLS 1.3). Ein ausreichend leistungsfähiger Quantencomputer könnte viele heute verbreitete Public-Key-Verfahren angreifen. Das Problem beginnt jedoch potenziell schon vor der Existenz eines solchen Computers. Denn verschlüsselter Netzwerkverkehr kann heute aufgezeichnet und möglicherweise Jahre später entschlüsselt werden („Harvest now, decrypt later"). Java 27 unterstützt deshalb drei hybride TLS-1.3-Gruppen, wie zum Beispiel X25519MLKEM768. Dabei werden ein klassischer elliptischer Schlüsselaustausch und das quantenresistente ML-KEM kombiniert. Der hybride Ansatz ist wichtig. Denn Java setzt nicht einfach alles auf den vergleichsweise jungen Post-Quantum-Algorithmus. Stattdessen ist die Verbindung sicher, solange mindestens einer der beiden beteiligten Algorithmen sicher bleibt. Für Anwendungen ist das weitgehend transparent. Bestehender JSSE-, HTTPS- oder HTTP-Client-Code muss nicht auf eine neue API umgestellt werden. Unterstützen beide Seiten eine hybride Gruppe, kann sie während des TLS-Handshakes ausgehandelt werden. Damit gehört Post-Quantum TLS zu den relevanten Neuerungen des Jahres, obwohl die meisten Java-Entwickler dafür keine einzige neue Codezeile schreiben werden.

Ebenfalls unter das Thema Sicherheit fällt der JEP 536 (JFR In-Process Data Redaction). Durch den Java Flight Recorder (JFR) können auch System Properties, Umgebungsvariablen und Kommandozeilenargumente aufgezeichnet werden. Darunter befinden sich in realen Anwendungen nicht selten Passwörter, Tokens oder andere Secrets. Bisher konnten JFR-Dateien nur nachträglich bereinigt werden. Dann waren die sensiblen Daten allerdings bereits im Umlauf. Ab Java 27 werden solche Informationen deshalb schon innerhalb der JVM geschwärzt, bevor sie in das Recording gelangen. Potenziell sensible Informationen werden durch einen Platzhalter ersetzt. Gerade weil JFR-Dateien häufig zwischen Produktion, Entwicklung und externem Support ausgetauscht werden, ist das ein sinnvoller Schritt hin zu sichereren Defaults.

Standardmäßig werden Inhalte geschwärzt, die vordefinierten Patterns (*secret*, *pwd*, *password*, *api*key*, …) entsprechen. Beim Starten der Anwendung können diese Muster erweitert oder durch eigene ersetzt werden. Dabei wird nach Mustern für Umgebungsvariablen und System Properties sowie für Programmargumente unterschieden. Zudem ist es auch möglich, die Schwärzung komplett zu deaktivieren *(siehe Listing 11)*.

```
java -XX:FlightRecorderOptions:'redact-
key=none,redactargument:none' ...
```

Listing 11: Schwärzung der Geheimnisse durch JFR deaktivieren

`final` soll künftig wirklich final bedeuten

Auch außerhalb klassischer Kryptografie verfolgt Java das Thema Security mit Integrity by Default weiter. Eigentlich sollte ein final-Feld nach seiner Initialisierung nicht mehr verändert werden können. Deep Reflection kann diese Garantie aber in bestimmten Situationen umgehen. Und insbesondere Bibliotheken und Java Tools machen davon viel Gebrauch. Auch Entwickler könnten sich diese Tricks jederzeit zunutze machen. Der JVM entgehen durch diese Optionen aber Möglichkeiten der tiefgreifenden Optimierung. Der JEP 500 (Prepare to Make Final Mean Final) ändert ab Java 26 zwar noch nicht viel. Aber es werden durch die JVM Warnungen erzeugt, wenn Reflection ein final-Feld nachträglich verändert.

Für die Übergangszeit existieren Optionen wie `--enable-final-field-mutation`. Wichtig dabei ist jedoch die Richtung. Nicht eine Bibliothek soll sich selbst diese Fähigkeit geben können, sondern die Anwendung beziehungsweise deren Betreiber muss sie bewusst erlauben. Das verbessert nicht nur die Integrität von Objekten. Wenn die JVM sich tatsächlich darauf verlassen kann, dass ein final-Feld unveränderlich bleibt, eröffnet dies auch zusätzliche Optimierungsmöglichkeiten.

Applets sind endgültig Geschichte

Ein Stück Java-Geschichte endet im Jahr 2026. Der JEP 504 hat die Applet API vollständig entfernt. Das betrifft insbesondere das Package java.applet sowie die damit verbundenen APIs in AWT, Swing und JavaBeans. Browser-Plug-ins für Java spielen schon seit vielen Jahren keine praktische Rolle mehr. Die API war seit Java 9 deprecated und seit Java 17 zur Entfernung vorgesehen. Java 26 vollzieht nun den letzten Schritt. Der API-Vergleich zwischen Java 25 und 27 *[4]* zeigt entsprechend die Löschung von JApplet und AppletInitializer. Für moderne Anwendungen dürfte das kaum Konsequenzen haben. Die Entfernung ist vor allem ein gutes Beispiel dafür, dass Java trotz seiner ausgeprägten Rückwärtskompatibilität historische APIs nicht unbegrenzt mitführt.

Vector API: Warten auf Valhalla

Eine fast schon traditionelle Position auf der JEP-Liste nimmt die Vector API ein. Java 26 bringt die elfte, Java 27 die zwölfte Incubator-Ver-

sion. Die API ermöglicht explizite SIMD-Berechnungen *(siehe Listing 12)*. SIMD steht für Single Instruction, Multiple Data und die JVM kann solche Operationen auf Prozessor-Instruktionsbefehle wie AVX oder NEON abbilden und damit die Berechnung deutlich beschleunigen.

```
var va = FloatVector.fromArray(SPECIES, a, offset);
var vb = FloatVector.fromArray(SPECIES, b, offset);
va.add(vb).intoArray(result, offset);
```

Listing 12: Beispiel der Vector API

In beiden JEPs gibt es kaum inhaltliche Veränderungen. Der Grund für die lange Inkubation liegt vor allem bei Project Valhalla und der Einführung von Value Classes. Denn Vektoren sind typische Value Objects. Sie brauchen keine Identität und sollten möglichst ohne klassischen Objekt-Overhead verarbeitet werden. OpenJDK möchte deshalb zunächst die entsprechenden Value-Class-Funktionen aus Valhalla abwarten und die Vector API anschließend darauf aufbauen, statt heute eine API zu standardisieren, die kurze Zeit später grundlegend geändert werden müsste. Im März 2027 sollen Value Classes in Java 28 übrigens als Preview Einzug halten. Ab dann werden wir vermutlich auch Previews und hoffentlich bald auch eine finale Version der Vector API sehen.

Kleine Verbesserungen in den APIs

Nicht jede interessante Änderung bekommt einen JEP. Die Release Notes *([5]* und *[6])* sowie der API-Vergleich des Java Almanac *[4]* zeigen aber noch viele kleinere Ergänzungen, die im Entwickleralltag durchaus nützlich sind. So ergänzt Java 26 die Klasse String um `text.equalsFoldCase(other)` und `text.compareToFoldCase(other)` sowie den Comparator um `String.UNICODE_CASEFOLD_ORDER`. Case Folding ist für Unicode-definierte Vergleiche gedacht, bei denen Groß- und Kleinschreibung ignoriert werden sollen. Es ist robuster als beide Strings zunächst mit `toLowerCase()` zu transformieren. Auch in Java 27 wird die String-Klasse ergänzt. Mit `text.encodedLength(StandardCharsets.UTF_8)` lässt sich feststellen, wie viele Bytes ein String in einer bestimmten Kodierung benötigt, ohne zunächst über `text.getBytes(StandardCharsets.UTF_8)` ein byte-Array erzeugen zu müssen. Passend dazu hat auch die Foreign Function & Memory API neue String-Operationen auf MemorySegment und SegmentAllocator erhalten.

Bereits in Java 26 wird die Klasse Process von AutoCloseable und Closeable abgeleitet. In einem Try-with-Resources-Block wird beim Schließen sichergestellt, dass der Prozess beendet ist und die verbundenen Streams und Ressourcen freigegeben werden *(siehe Listing 13)*.

```
try (Process process =
        new ProcessBuilder("some-command").start()) {
    // ...
}
```

Listing 13: Process in Try-with-Resources

Ebenfalls in Java 26 wurden der Klasse Comparator zwei Convenience-Methoden (`comparator.min(a, b)` und `comparator.max(a, b)`) hinzugefügt. Damit muss für die Auswahl des kleineren oder größeren Elements nicht mehr selbst `compare()` ausgewertet werden.

Die Java Database Connectivity API (JDBC) wurde in Java 26 auf die Version 4.5 aktualisiert. Neu sind unter anderem die SQL-Typen `JDBCType.JSON` und `JDBCType.DECFLOAT`. Außerdem implementieren Array, Blob, Clob, NClob und SQLXML nun AutoCloseable. Damit kann beispielsweise ein Blob direkt mit Try-with-Resources verwendet werden, wodurch beim Verlassen des Blocks das Freigeben der Ressourcen sichergestellt wird.

In Java 27 wurden die Klassen Math und `StrictMath` um die inversen hyperbolischen Funktionen für Sinus, Cosinus und Tangens (`Math.asinh(x)`, `Math.acosh(x)`, `Math.atanh(x)`) erweitert. BigDecimal bekommt zusätzlich die Methode `rootn(n, mathContext)` für die Berechnung einer n-ten Wurzel. Und die vordefinierten DateTimeFormatter akzeptieren nun auch kurze ISO-8601-Offsets wie „2026-09-01T10:00+02" und nicht mehr nur Varianten wie 2026-09-01T10:00+02:00. Das ist vor allem beim Einlesen externer ISO-8601-Daten praktisch.

Außerdem räumt Java weiter auf. Neben der Applet API verschwinden weitere alte Mechanismen. Dazu gehören `Thread.stop()` sowie mehrere alte Netzwerkmethoden und außerdem `ThreadPoolExecutor.finalize()`. Auch beim Java Launcher wurden alte Optionen (`-noverify`, `-Xverify:none`, `-verifyremote`, `-noclassgc`) entfernt. Insbesondere die Möglichkeit, die Bytecode-Verifikation vollständig abzuschalten, verschwindet damit. Solche Änderungen zeigen eine Entwicklung, die sich durch viele aktuelle Java-Releases zieht. Java behält seine starke Rückwärtskompatibilität, trägt aber nicht mehr jede historische Ausnahme unbegrenzt weiter.

Fazit und Ausblick

Die Neuerungen von Java 26 und 27 wirken auf den ersten Blick weniger spektakulär als Releases, die beispielsweise Records, Sealed Classes oder Virtual Threads eingeführt haben. Betrachtet man beide Versionen gemeinsam, wird jedoch eine klare Entwicklung sichtbar. Java verbessert weiter sein Programmiermodell. Pattern Matching wird auf primitive Typen ausgeweitet, Structured Concurrency ergänzt Virtual Threads um eine strukturierte Behandlung zusammengehöriger Tasks und Lazy Constants schließen die Lücke zwischen verzögerter Initialisierung und Unveränderlichkeit.

Gleichzeitig entwickelt sich die Laufzeitumgebung weiter. G1 wird schneller und zum einheitlichen Default, Compact Object Headers reduzieren den Speicherverbrauch und Project Leyden arbeitet weiter daran, Start- und Warmup-Zeiten zu verkürzen. Ein besonders wichtiges Thema ist Security. Eine standardisierte PEM API beseitigt eine lange bestehende Lücke, TLS bereitet sich mit hybriden Verfahren auf Post-Quantum-Kryptografie vor, der Java Flight Recorder schützt Secrets besser und Java schränkt mit „Make Final Mean Final" schrittweise Mechanismen ein, die zentrale Garantien der Sprache umgehen. Mit HTTP/3 und zahlreichen kleineren Verbesserungen an Strings, JDBC, Prozessen, Mathematik und anderen APIs kommen außerdem Funktionen hinzu, die sich unmittelbar im Anwendungscode nutzen lassen.

Java 26 und 27 stehen damit weniger für einzelne große Neuerungen, sondern für die konsequente Weiterentwicklung der Plattform

auf vielen Ebenen gleichzeitig: Sprache, Nebenläufigkeit, Performance, Startup, Netzwerk, Sicherheit und Klassenbibliothek. Während neue Möglichkeiten hinzukommen, verschwinden gleichzeitig historische Altlasten und Sonderwege. Und richtig spannend wird es im nächsten Jahr, wenn mit dem JEP 401 nach mehr als 10 Jahren Entwicklung endlich Value Classes/Objects Einzug ins OpenJDK erhalten. Seit dem Feature Freeze vom OpenJDK 27 (Rampdown Phase One am 04.06.2027) sind die ersten Veränderungen im Git Repo zu sehen. Die Ankündigung auf der Mailingliste *[7]* zeigt einige Details. Value Classes sind die größte Veränderung des Java-Typsystems seit Generics. Die darauf aufbauenden JEPs (Enhanced Primitive Boxing, Null-Restricted and Nullable Types und Null-Restricted Value Class Types) und weitere Ideen können auf dem JEP-Index *[8]* nachgelesen werden. Nach einigen ruhigeren Releases wird es mit dem OpenJDK 28 wieder spektakuläre neue Features geben. Uns wird als Java-Entwicklern in den nächsten Jahren also nicht langweilig werden.

Referenzen:

[1] *https://openjdk.org/projects/jdk/*
[2] *https://openjdk.java.net/projects/jdk/26/*
[3] *https://openjdk.java.net/projects/jdk/27/*
[4] *https://javaalmanac.io/jdk/27/apidiff/25/*
[5] *https://jdk.java.net/26/release-notes*
[6] *https://jdk.java.net/27/release-notes*
[7] *https://mail.openjdk.org/archives/list/jdk-dev@openjdk.org/thread/AIA3O3LHFZ6T7TIPH7KZT4WS4B6U72U5/*
[8] *https://openjdk.org/jeps/0#Draft-and-submitted-JEPs*



Falk Sippach
falk@jug-da.de
https://twitter.com/sippsack

Falk Sippach ist bei der embarc Software Consulting GmbH als Softwarearchitekt, Berater und Trainer stets auf der Suche nach dem Funken Leidenschaft, den er bei seinen Teilnehmern, Kunden und Kollegen entfachen kann. Bereits seit über 20 Jahren unterstützt er in meist agilen Softwareentwicklungsprojekten im Java-Umfeld. Als aktiver Bestandteil der Community (Mitorganisator der JUG Darmstadt) teilt er zudem sein Wissen gern in Artikeln, Blog-Beiträgen, sowie bei Vorträgen auf Konferenzen oder User Group Treffen und unterstützt bei der Organisation diverser Fachveranstaltungen. Falk twittert unter @sippsack.

MITMACHEN UND AUTORIN ODER AUTOR WERDEN!



Du kennst dich in einem bestimmten Gebiet aus dem Java-Themenbereich aus und du möchtest als Autorin oder Autor für die Java aktuell dein Wissen mit der Community teilen?

Nimm Kontakt zu uns aufund sende deinen Artikelvorschlag an:
redaktion@java-aktuell.eu
Wir freuen uns, von dir zu hören!

Weitere Informationen